

Finite-Volume Simulation of the 1D Sod Shock Tube Problem

Godunov-Type, MUSCL and Hybrid Shock-Capturing Schemes in Fortran 90

Author	Mohammad Shahid
Project type	Numerical methods / CFD portfolio project
Language	Fortran 90 with Python post-processing
Test case	Classical 1D Sod shock tube problem
Final time	$t = 0.25$ s
Meshes	$n_x = 201$ and $n_x = 1001$

Abstract

This report documents a cleaned and GitHub-ready implementation of a one-dimensional finite-volume solver for the compressible Euler equations. The code solves the Sod shock tube problem using a first-order Godunov-type finite-volume scheme with the local Lax-Friedrichs/Rusanov approximate Riemann flux, a MUSCL reconstruction with the Van Albada limiter, and a simple hybrid option that reverts to first-order interface states near strong pressure jumps. The numerical results are compared with the exact Riemann solution for density, velocity and pressure at two mesh resolutions. The purpose of the project is to present a compact, reproducible and well-documented CFD/numerical-methods example suitable for a public technical portfolio.

Important note: in this cleaned code, the label "godunov" refers to a first-order Godunov-type finite-volume update using a Rusanov approximate flux. It is not an exact Godunov Riemann solver.

Keywords: finite volume method, Sod shock tube, Euler equations, Godunov-type method, Rusanov flux, MUSCL reconstruction, Van Albada limiter, Fortran 90, CFD.

1. Introduction

Shock-tube problems are standard benchmark tests for compressible-flow solvers because they contain the main wave features of hyperbolic conservation laws: rarefaction waves, contact discontinuities and shock waves. The Sod shock tube is especially useful because it has a known exact Riemann solution, allowing direct comparison between numerical and analytical profiles.

The original assignment compared velocity, density and pressure profiles against the exact solution for mesh sizes of 201 and 1001 points at $t = 0.25$ s. The cleaned GitHub version keeps the same numerical objective, but reorganizes the code into a reproducible project with a single solver, command-line options, CSV output, Python plotting and a technical methodology document.

2. Governing Equations

The governing equations are the one-dimensional compressible Euler equations for an inviscid ideal gas. In conservative form, they are written as:

$$dU/dt + dF(U)/dx = 0$$

where the conserved variable vector U and flux vector $F(U)$ are:

$$U = [\rho, \rho u, E]^T$$

$$F(U) = [\rho u, \rho u^2 + p, u(E + p)]^T$$

Here ρ is density, u is velocity, p is pressure and E is total energy per unit volume. The ideal-gas closure is:

$$E = p/(\gamma - 1) + 0.5 \rho u^2, \text{ with } \gamma = 1.4$$

The system is hyperbolic for positive density and pressure. The characteristic wave speeds are $u - a$, u and $u + a$, where $a = \sqrt{\gamma p/\rho}$ is the speed of sound.

3. Sod Shock Tube Problem

The computational domain is x in $[0, 1]$. A diaphragm initially separates two constant states at $x = 0.5$. At $t = 0$ the diaphragm is removed and the discontinuity evolves into a rarefaction wave moving left, a contact discontinuity moving right, and a shock moving further right.

Quantity	Left state, $x < 0.5$	Right state, $x \geq 0.5$
Density, ρ	1.0	0.125
Velocity, u	0.0	0.0
Pressure, p	1.0	0.1
Specific heat ratio, γ	1.4	1.4

Zero-gradient/transmissive boundary conditions are applied at both ends of the domain. The simulation is run to $t = 0.25$ s for $n_x = 201$ and $n_x = 1001$ grid points.

4. Numerical Method

4.1 Finite-volume discretization

The domain is divided into control volumes. The finite-volume update for cell i is:

$$U_i^{(n+1)} = U_i^{(n)} - (\Delta t / \Delta x) (F_{\{i+1/2\}} - F_{\{i-1/2\}})$$

The time step is selected adaptively using a CFL condition:

$$\Delta t = \text{CFL } \Delta x / \max_i (|u_i| + a_i), \text{ with CFL} = 0.5$$

4.2 Godunov-type flux used in the code

The first-order option uses the local Lax-Friedrichs flux, commonly called the Rusanov flux. This is a Godunov-type approximate Riemann flux because it computes a stable interface flux from left and right states without solving the exact Riemann problem.

$$F_{\{i+1/2\}} = 0.5(F_L + F_R) - 0.5 \alpha (U_R - U_L)$$

where $\alpha = \max(|u_L| + a_L, |u_R| + a_R)$. This flux is robust near shocks, but it is relatively diffusive, especially near contact discontinuities.

4.3 MUSCL reconstruction

The MUSCL option improves the spatial reconstruction by building left and right states at each interface from limited slopes. The present implementation uses a Van Albada limiter to reduce non-physical oscillations near discontinuities while allowing sharper gradients in smoother regions.

$$r = (U_i - U_{i-1}) / (U_{i+1} - U_i)$$
$$\text{phi}(r) = (r^2 + r) / (r^2 + 1) \text{ for } r > 0, \text{ otherwise } \text{phi}(r) = 0$$

Only the spatial reconstruction is made higher resolution. Since the time integration remains forward Euler, this compact educational implementation should not be presented as a fully second-order accurate production solver. The reconstructed states are checked for positive density and pressure. If a reconstructed state is not physically admissible, the code falls back locally to the original cell-average state.

4.4 Hybrid scheme

The hybrid option applies MUSCL reconstruction in smooth regions and reverts to first-order interface states near strong pressure jumps. A simple pressure-gradient sensor is used:

$$\text{sensor} = |p_R - p_L| / \max(|p_R + p_L|, \text{epsilon})$$

If the sensor is larger than the selected threshold, the interface is treated with first-order states. This provides extra robustness around shocks at the cost of local numerical diffusion.

5. Implementation and Repository Features

The GitHub version is organized as a reproducible numerical-methods project. Instead of presenting the work only as an assignment submission, the repository provides a single solver, documented run commands, reusable output files and plotting scripts. The main implementation features are summarized below.

Area	Repository implementation
Solver interface	One Fortran 90 command-line solver with selectable Godunov-type/Rusanov, MUSCL and hybrid options.
Numerical flux	Local Lax-Friedrichs/Rusanov approximate Riemann flux used consistently at cell interfaces.
MUSCL reconstruction	Limited slope reconstruction with Van Albada limiter and small-denominator protection in the slope-ratio calculation.
Positivity handling	Reconstructed interface states are checked for positive density and pressure; non-admissible states fall back locally to first-order states.
Conservative variables	Density is taken directly from the conservative state vector, avoiding divisions by velocity when velocity is close to zero.
Time stepping	Adaptive CFL-based time step based on the maximum local wave speed $ u + a$.
Output data	Simulation results are written as clean CSV files containing x, density, velocity, pressure and energy.
Reproducibility	Makefile, README, methodology notes and Python plotting script are included for repeatable runs.

6. Results and Discussion

The following figures compare density, velocity and pressure profiles produced by the Godunov-type/Rusanov, MUSCL and hybrid schemes against the exact solution. Results are shown for $n_x = 201$ and $n_x = 1001$ at $t = 0.25$ s.

6.1 Mesh size $n_x = 201$

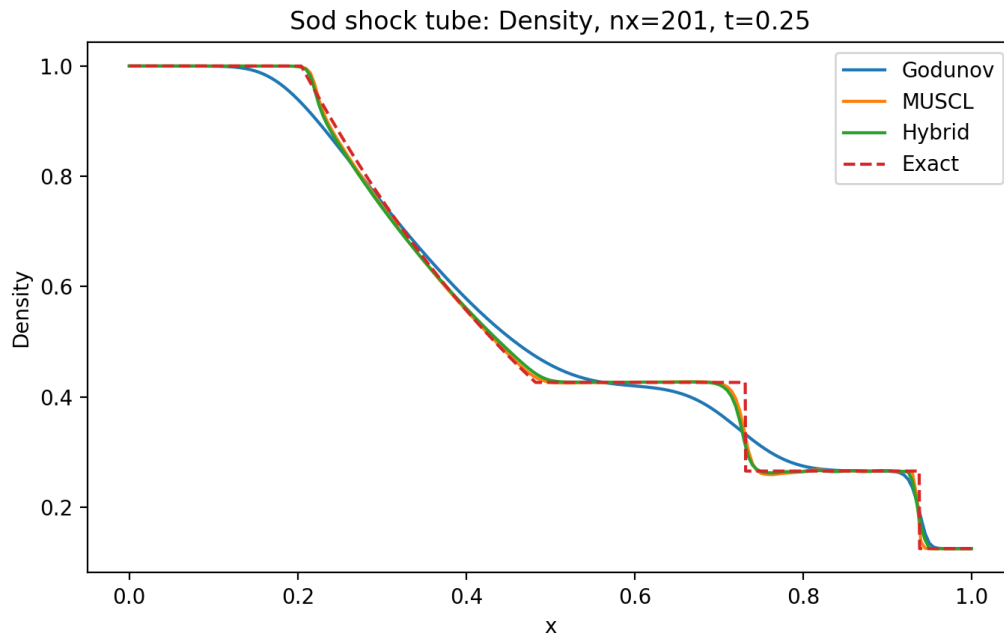


Figure 1. Density comparison for $n_x = 201$ at $t = 0.25$ s.

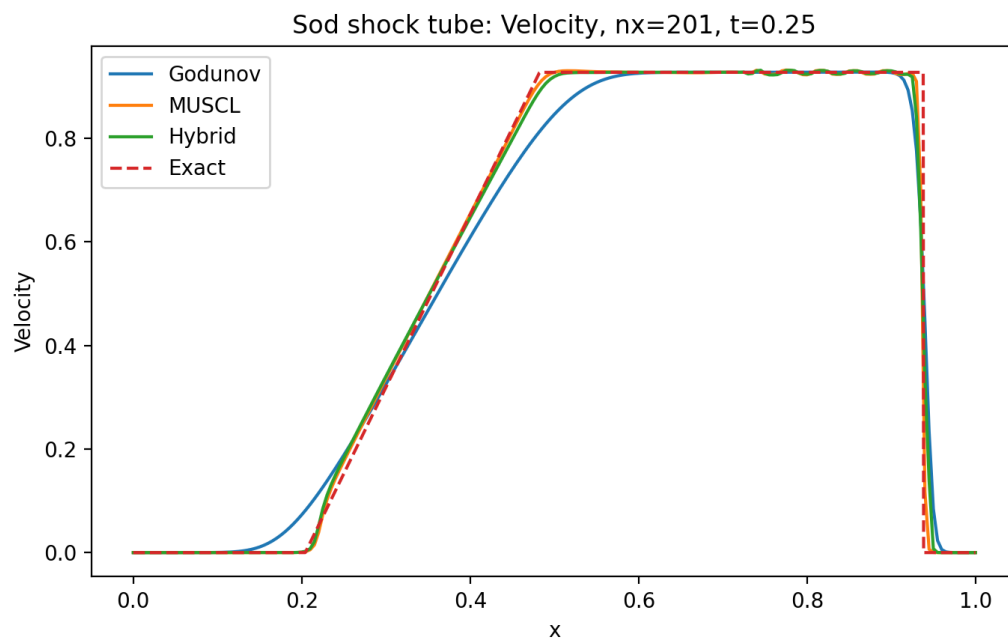


Figure 2. Velocity comparison for $n_x = 201$ at $t = 0.25$ s.

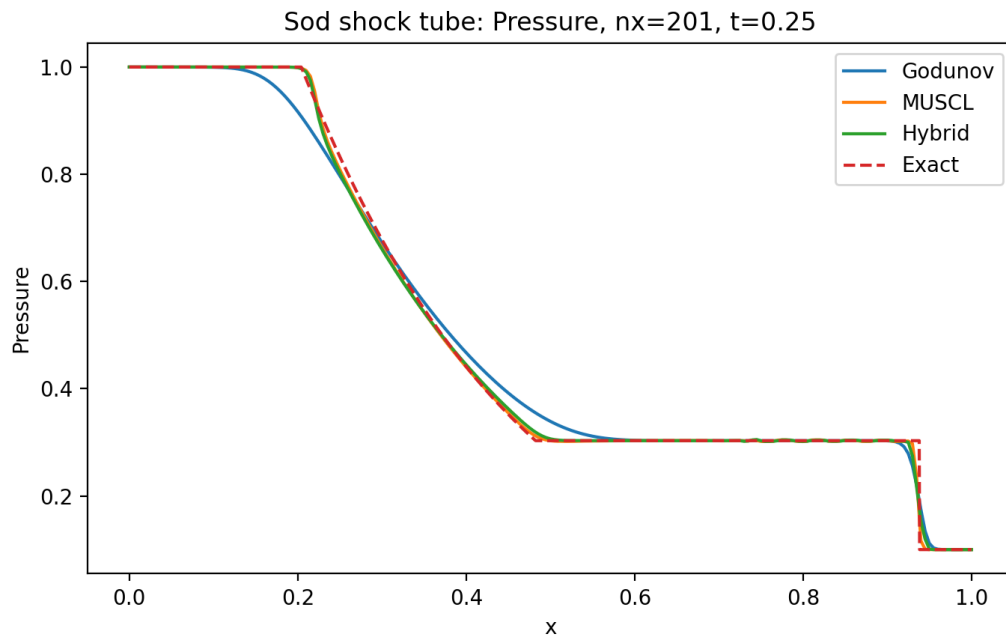


Figure 3. Pressure comparison for $nx = 201$ at $t = 0.25$ s.

For $nx = 201$, all schemes reproduce the main wave pattern: left rarefaction, contact discontinuity and right shock. The first-order Godunov-type/Rusanov scheme is more diffusive, while MUSCL better resolves gradients. The hybrid method remains stable near strong pressure gradients and can reduce oscillatory behavior around discontinuities.

6.2 Mesh size $n_x = 1001$

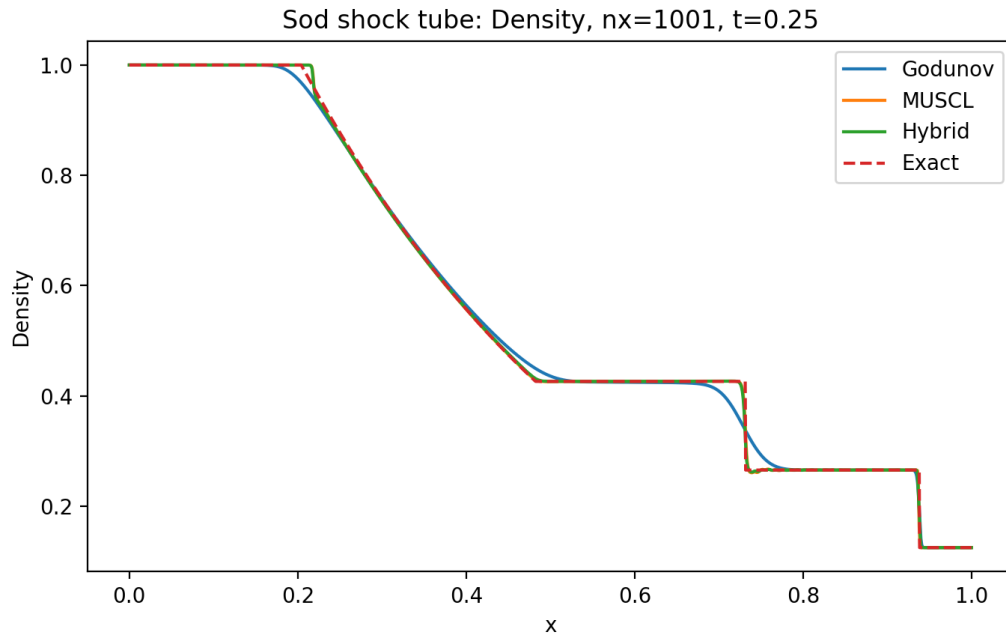


Figure 4. Density comparison for $n_x = 1001$ at $t = 0.25$ s.

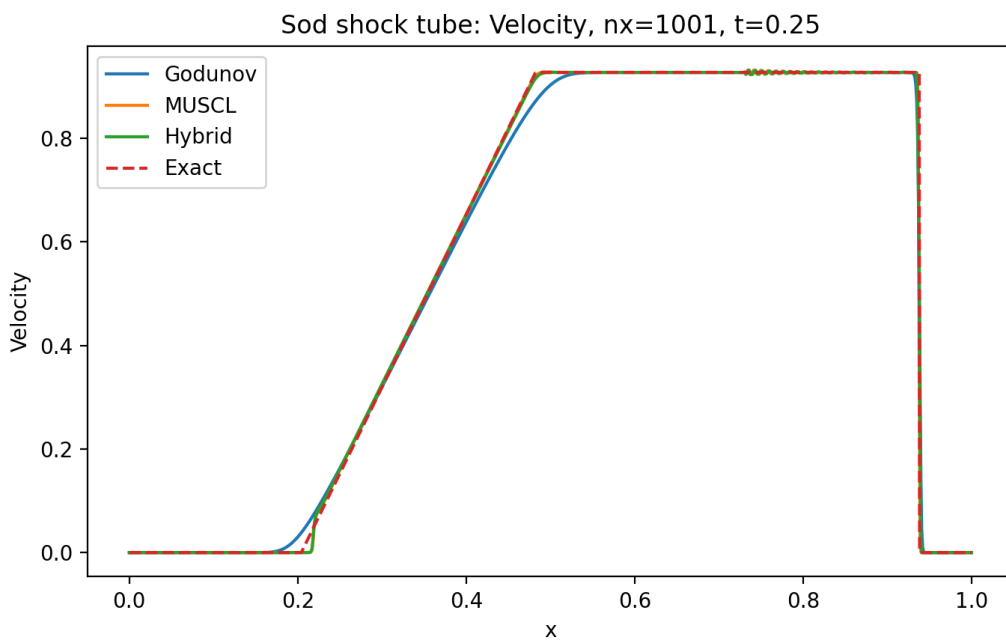


Figure 5. Velocity comparison for $n_x = 1001$ at $t = 0.25$ s.

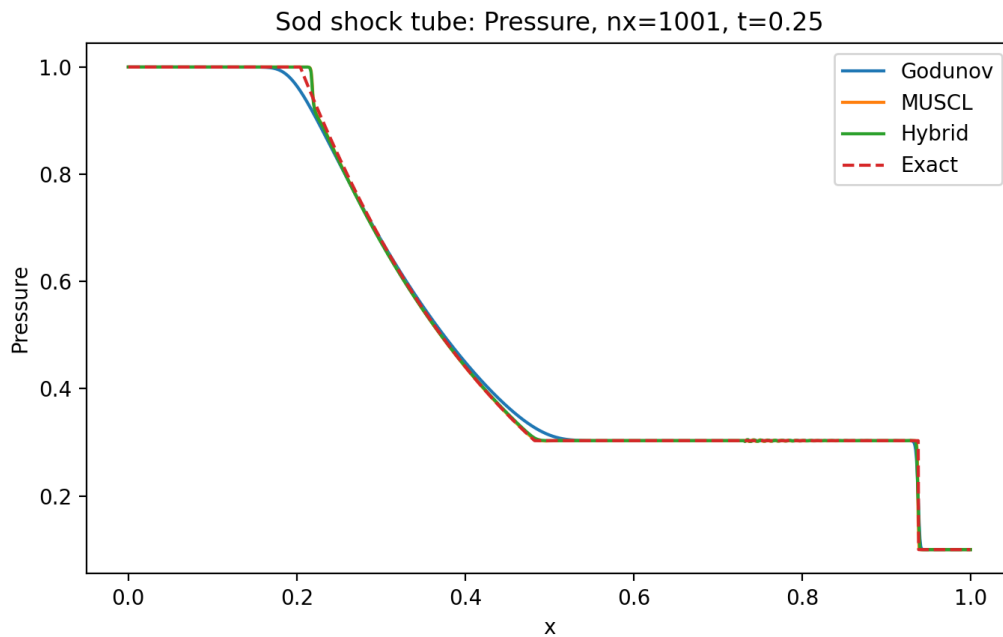


Figure 6. Pressure comparison for $nx = 1001$ at $t = 0.25$ s.

With $nx = 1001$, the numerical profiles become sharper and the agreement with the exact solution improves. The shock and contact discontinuity are captured over fewer grid cells compared with the coarser grid. This demonstrates the expected mesh-refinement behavior for finite-volume shock-capturing schemes.

7. Repository Organization

The GitHub-ready project is organized as follows:

```
finite-volume-shock-tube-fortran/ README.md Makefile src/shock_tube_solver.f90
scripts/plot_comparison.py results/*.csv figures/*.png docs/methodology.md
report/shock_tube_project_report.pdf legacy_original/*.f90
```

The original assignment PDF and original Fortran files are preserved for traceability, while the cleaned implementation is placed in `src/` and documented through `README.md`, `docs/methodology.md` and this report.

8. Limitations and Possible Future Work

The present implementation is intentionally compact and educational. It uses a Rusanov numerical flux rather than an exact Riemann solver, HLLC flux or Roe flux. Rusanov flux is robust but can be more diffusive, especially around contact waves. Future improvements could include HLL, HLLC or Roe fluxes, second-order time integration, a formal error-convergence table, automated unit tests and comparison against additional shock-tube cases.

9. Conclusion

This project demonstrates a clean finite-volume implementation of the 1D Sod shock tube problem in Fortran 90. It includes a first-order Godunov-type/Rusanov option, MUSCL reconstruction and a hybrid reconstruction option, adaptive CFL stepping, physical-state checks and exact-solution comparison plots. After cleanup, the project is suitable for inclusion in a GitHub portfolio as evidence of numerical-methods, CFD and scientific-computing experience.

References

- [1] Godunov, S. K. (1959). A difference method for numerical calculation of discontinuous solutions of the equations of hydrodynamics. *Matematicheskii Sbornik*, 47(89), 271-306.
- [2] Sod, G. A. (1978). A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws. *Journal of Computational Physics*, 27(1), 1-31. [https://doi.org/10.1016/0021-9991\(78\)90023-2](https://doi.org/10.1016/0021-9991(78)90023-2)
- [3] van Leer, B. (1979). Towards the ultimate conservative difference scheme. V. A second-order sequel to Godunov's method. *Journal of Computational Physics*, 32(1), 101-136. [https://doi.org/10.1016/0021-9991\(79\)90145-1](https://doi.org/10.1016/0021-9991(79)90145-1)
- [4] van Albada, G. D., van Leer, B., & Roberts, W. W., Jr. (1982). A comparative study of computational methods in cosmic gas dynamics. *Astronomy and Astrophysics*, 108, 76-84.
- [5] Rusanov, V. V. (1962). The calculation of the interaction of non-stationary shock waves and obstacles. *USSR Computational Mathematics and Mathematical Physics*, 1(2), 304-320. [https://doi.org/10.1016/0041-5553\(62\)90062-9](https://doi.org/10.1016/0041-5553(62)90062-9)
- [6] Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*. 3rd ed. Springer. <https://doi.org/10.1007/b79761>
- [7] LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511791253>
- [8] Original assignment material: Mohammad Shahid, Assignment No. 5, Fortran (F90) Code on 1D Shock Tube Problem, submitted to Prof. M. F. S. Baig.